HOMEHUNT

Main Project Report

Submitted By

ARUN PS :230021080343

In partial fulfilment of the requirements for award of the degree of

BACHELOR OF COMPUTER APPLICATION

MAHATMA GANDHI UNIVERSITY, KOTTAYAM–686560

MAR ELIAS COLLEGE, KOTTAPPADY P.O.-686692

(Affiliated to Mahatma Gandhi University, Kottayam-686560)

2023-2026

MAR ELIAS COLLEGE KOTTAPPADY

(Affiliated to Mahatma Gandhi University, Kottayam-686560)
(Accredited by NAAC with B++ Grade

CERTIFICATE

This is to certify that the project report titled **HOMEHUNT** submitted by **ARUN PS (230021080343)** towards partial fulfilment of the requirement for the award of the degree of BACHELOR OF COMPUTER APPLICATION is a record of Bonafide work carried out by him during the academic year 2025 - 2026.

Ms. Ashlin K G	**Ms.Poornima N**	**Dr.Joseph T.Moolayil**
ProjectGuide	HOD	Principal

Submitted for Viva Voce held on…….

Internal Examiner **External Examiner**

DECLARATION

DECLARATION

I, hereby declare that this project report entitled **HOMEHUNT** has been written and submitted to the department of Computer Science and Applications, Mar Elias College, Kottappady in partial fulfilment of the award of the degree of Bachelor of Computer Application is an authentic record of my original work. The empirical findings in the report are on information collected by me and are not copied from anywhere.

Arun p s

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

My endeavour stands incomplete without dedicating my gratitude to few people who have contributed a lot towards the successful completion of my project work.

To bring out something successful is truly the work of God. I am indebted to God Almighty for being the guiding light throughout the project.

I express my sincere thanks to Dr. Joseph T. Moolayil, Principal, Mar Elias College, for providing necessary facilities.

I also thank Ms. Poornima N, HOD, Department of Computer Science and Applications.
I sincerely express gratitude to her,

I also thank Ms. Ashlin K G, my project guide for innumerable acts of timely advice, encouragement and I sincerely express my gratitude to her.

I also thank Mr. Abi V J, my project guide of progressive project center
for innumerable acts of timely advice, encouragement and I sincerely express my gratitude to him.

I owe my thanks to my friends and all others who have directly or indirectly helped in the successful completion of this project.

No words can express my humble and eternal gratitude to my beloved parents and relatives who has been guiding me in all walks of my life.

Arun p s

ABSTRACT

HomeHunt is a web-based rental house management system developed using React, HTML, CSS, and JavaScript for the frontend and Django with SQLite for the backend. It is an interactive digital platform designed to simplify the process of rental house discovery and management. The platform connects house owners and tenants through a centralized system where rental properties can be listed, searched, and managed efficiently. Owners can upload houses with complete details including rent amount, BHK type, floor type, location, tenant preference, and property images. Users can browse available houses, search using filtering options, send rental requests, and make advance payments through the platform. The admin module provides complete system control by managing users, owners, houses, complaints, and feedback submitted by users. The system also includes features such as gallery management, complaint handling, and feedback submission. With its user-friendly interface and structured workflow, HomeHunt simulates a real-world rental house management platform where property owners can publish houses and tenants can easily discover suitable rental homes.

MODULES

Admin:

- **Platform Oversight:** Monitor and control all system activities
- **Owner Management:** Verify and manage property owner accounts.
- **User Management:** View and manage registered users in the platform
- **House Management: Monitor and manage house listings uploaded by owners.**
- **Complaint Handling:** View and respond to complaints submitted by users.
- **Feedback Monitoring:** Review feedback submitted by users to improve system performance
- **System Control:** Maintain the overall functionality and security of the platform.

Owner:

- **Property Gallery:** Upload and manage multiple images of houses.
- **Response Management:** Accept or reject rental requests from users.
- **Property Monitoring:** Track the status and availability of listed houses.
- **Request Management:** View rental requests sent by users.
- **Chat Communication**: Communicate with users regarding rental requests through a real-time chat interface.
- **House Management:** Upload, edit, and manage rental house listings with details and images.

USER:

- **Filter Options:** Filter houses by location, price, BHK type, and floor type.
- **House Search:** Browse and search available rental houses
- **Request Submission:** Send rental requests to house owners.
- **Favorites Management:** Add and manage favorite houses.
- **Complaint System:** Submit complaints to the administrator.
- **Feedback System:** Provide feedback about the system and services.
- **Chat with Owner**: Users can communicate directly with property owners after sending rental request.

FRONTEND

- HTML
- React
- CSS
- JavaScript

BACKEND

- Python
- Django
- SQLite

CONTENTS

1. INTRODUCTION

INTRODUCTION

This system is developed as a comprehensive web platform for rental house discovery and management using React, HTML, CSS, and JavaScript for the frontend interface while Django with SQLite powers the backend operations. The frontend technologies provide a responsive and user-friendly interface that allows users to search and browse rental houses efficiently. Django handles core functionalities including user authentication, property management, request processing, and complaint handling. SQLite efficiently stores and manages all essential data such as user profiles, owner details, house listings, gallery images, rental requests, and payments. The HomeHunt system simplifies the process of finding rental houses by providing a centralized platform where owners can publish their properties and tenants can search houses according to their preferences. The system also ensures secure user authentication and efficient data management, making it a reliable solution for modern rental house discovery.

MODULES

1. Admin
2. Owner
3. User

1.1 OVERVIEW OF THE PROJECT

HomeHunt is a web application developed using React, HTML, CSS, and JavaScript for the frontend interface while Django and SQLite power the backend operations. The proposed system consists of three main modules: Admin, Owner, and User. Each module is designed with specific functionalities that allow users to search rental houses, owners to manage property listings, and administrators to monitor and control the system activities, The system also includes a communication feature that allows tenants and property owners to interact through a chat interface

1.2 ABOUT THE ORGANIZATION

HomeHunt is an innovative web platform that connects house owners with tenants through a centralized rental management system. The platform simplifies the process of searching rental houses by providing filtering options based on location, rent amount, BHK type, and floor type. Owners can easily publish property listings and manage rental requests while users can explore available houses and send requests directly through the system. Through its user-friendly interface and efficient management system, HomeHunt transforms the traditional rental search process into a modern and organized digital platform.

1.SYSTEM SPECIFICATION

SYSTEM SPECIFICATION

ABOUT THE HARDWARE

The project under consideration requires the following specifications:

- Processor: Intel®Corei3Processor
- Memory:4GB
- Keyboard: Any QWERTY Keyboard
- Mouse: Any Optical mouse
- Display: Any display that supports 1024x768 resolution
- USB: Any USB 2.0 or above supporting port

ABOUT THE SOFTWARE

The software for the development of the proposed system is as follows:

- Operating System: Windows 11 Home Single Language
- Web Server: Django Development Server
- Web Browser: Google Chrome
- Database: SQLite
- Languages: Python, HTML, CSS, JavaScript
- IDE: Visual Studio Code 1.62.2

DJANGO DEVELOPMENT

Django Development Server is the built-in web server provided by the Django framework for running web applications during development. It allows developers to test and run Django applications locally without the need for configuring external web servers.

The Django development server runs using the command python manage.py runserver, which starts the application on a local host and allows developers to access the system through a web browser.

Features of Django Development Server:

- Provides a simple environment for testing and debugging web applications.
- Automatically reloads the server when code changes are made.
- Allows developers to run the web application locally.
- Integrates seamlessly with Django backend functionality.
- Helps developers test database operations and API responses during development.

In the HomeHunt system, the Django development server is used to run the application locally and test all functionalities such as user registration, property listing, rental requests, and complaint management

PYTHON

Python is a high-level programming language widely used for web development, data analysis, and software development. It is known for its simple syntax and readability, making it one of the most popular programming languages in modern software development.

Python is used as the backend programming language for the HomeHunt system through the Django framework. It handles server-side operations such as user authentication, data processing, request handling, and database communication.

Advantages of Python:

- Supports rapid application development.
- Simple and easy-to-understand syntax.
- Large community support and extensive libraries.
- Highly compatible with web frameworks such as Django and Flask.
- Suitable for building scalable web applications.

Applications of Python:

- Software development
- Automation and scripting
- Artificial intelligence and machine learning
- Data processing and analysis
- Web application development

DJANGO

Django is a high-level Python web framework that enables developers to build secure and scalable web applications quickly. It follows the Model-View-Template (MVT) architecture, which separates application logic, user interface, and data management.

Django provides built-in features such as authentication systems, database management, and security protections, making it ideal for modern web application development.

Features of Django:

- Rapid web application development
- Secure authentication system
- Built-in database management through ORM
- Easy integration with frontend technologies

SQLITE

SQLite is a lightweight relational database management system that stores the entire database in a single file on the system. Unlike other database systems, SQLite does not require a separate server process, making it simple and easy to use.

SQLite is widely used for small to medium web applications and is the default database used in Django projects during development.

Features of SQLite:

- Reliable and efficient for web applications.
- Supports standard SQL queries.
- Stores the entire database in a single file.
- No separate database server required.
- Lightweight and easy to configure.

HTML

HTML stands for Hyper Text Markup Language, which is the most widely used language on Web to develop web pages. HTML was created by Berners-Lee in late 1991 but "HTML2.0" was the first standard HTML specification which was published in 1995. HTML 4.01 was a major version of HTML and it was published in late 1999. Though HTML 4.01 version is widely used but currently we are having HTML5 version which is an extension to HTML 4.01, and this version was published in 2012.

Originally, HTML was developed with the intent of defining the structure of documents like headings, paragraphs, lists, and so forth to facilitate the sharing of scientific information between researchers. Now, HTML is being widely used to format web pages with the help of different tags available in HTML language.

HTML is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning HTML:

- **Create Website** -You can create a website or customize an existing web template if you know HTML well.
- **Become a web designer**-If you want to start a career as a professional web designer, HTML and CSS designing is a must skill.
- **Understand web**-If you want to optimize your website, to boost its speed and performance, it is good to know HTML to yield best results.
- **Learn other languages** - Once you understand the basic of HTML then other related technologies like JavaScript, Python, or angular are become easier to understand.

Applications of HTML

As mentioned before, HTML is one of the most widely used language over the web. I' m going to list few of them here:

- **Webpages development**-HTML is used to create pages which are rendered over the web. Almost every page of web is having html tags in it to render its details in browser.
- **Internet Navigation**-HTML provides tags which are used to navigate fromone page to another and is heavily used in internet navigation.
- **Responsive UI** - HTML pages now-a-days works well on all platform, mobile, tabs, desktop or laptops owing to responsive design strategy.
- **Offline support**-HTML pages once loaded can be made available offline on the machine without any need of internet.
- **Game development**-HTML 5 has native support for rich experience and is now useful in gaming development arena as well.

CSS

CSS is used to control the style of a web document in a simple and easy way. CSS is the acronym for "Cascading Style Sheet". Cascading Style Sheets, fondly referred to as CSS, is a simple design language intended to simplify the process of making web pages presentable.

CSS is a and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning CSS:

- **Create Stunning Website** - CSS handles the look and feel part of a web page. Using CSS, you can control the colour of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colours are used, layout designs, variations in display for different devices and screen sizes as well as a variety of other effects.
- **Become a web designer**-If you want to start a career as a professional web designer, HTML and CSS designing is a must skill.
- **Control web** - CSS is easy to learn and understand but it provides powerful control over the presentation of an HTML document. Most commonly, CSS is combined with the markup languages HTML or XHTML.
- **Learn other languages**-Once you understand the basic of HTML and CSS then other related technologies like JavaScript or React are become easier to understand.

Applications of CSS

As mentioned before, CSS is one of the most widely used style language over the web. I'm going to list few of them here:

- **CSS saves time**-You can write CSS once and then reuse same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.
- **Pages load faster**-If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply it to all the occurrences of that tag. Soless code means faster download times.
- **Easy maintenance** - To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.

- **Superior styles to HTML**-CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.
- **Multiple Device Compatibility**-Style sheets allow content to be optimized for more than one type of device. By using the same HTML document, different versions of a website can be presented for handheld devices such as PDAs and cell phones or for printing.
- **Global web standards** - Now HTML attributes are being deprecated and it is being recommended to use CSS. So, it's a good idea to start using CSS in all the HTML pages to make them compatible to future browsers.

WINDOWS 11

Windows 11 is the latest operating system from Microsoft. Windows 11 combines a fresh modern design with enhanced productivity features, bringing together the best elements of previous versions while adding new capabilities for making it more user-friendly across desktops, laptops, tablets, and touch-enabled devices.

VISUAL STUDIOCODE

Visual Studio Code is a light weight but powerful source code editor which runs on your desktop and is available for Windows, macOS and Linux. It comes with built-in support for JavaScript, Type Script and Node.js and has a rich ecosystem of extensions for other languages (such as C++, C#, Java, Python, PHP, Go) and runtimes (such as .NET and Unity).

2. SYSTEMANALYSIS

SYSTEM ANALYSIS

System analysis involves studying the existing rental house searching process, identifying problems, and defining the requirements for the proposed system. The analysis phase focuses on understanding the current methods used by tenants and property owners to manage rental housing and identifying the limitations present in those systems. The objective of system analysis is to design an efficient and organized system that simplifies the rental house discovery process and improves communication between house owners and tenants.

3.1 EXISTING SYSTEM

Currently, rental house searching is commonly done through brokers, social media platforms, newspaper advertisements, and multiple online property websites. In many cases, tenants must contact several property owners individually to gather details about available houses. This process is time-consuming and often results in incomplete or outdated information about rental properties. Property owners also face difficulties in reaching the right tenants because there is no centralized platform dedicated to managing rental house listings effectively.

LIMITATIONS OF EXISTING SYSTEM

- Tenants must search through multiple platforms to find houses.
- Lack of a centralized system for managing rental houses.
- Limited filtering options for searching properties.
- Poor communication between tenants and house owners.
- Difficulty in verifying property details and availability.

3.2 FEASIBILTY STUDY

Feasibility study is conducted to evaluate whether the proposed system is practical and beneficial. The purpose of the feasibility study is to determine the technical, economic, and operational feasibility of implementing the system. The HomeHunt system has been analyzed under the following feasibility factors.

HOMEHUNT

1. Technical Feasibility

2. Behavioural Feasibility

3. Operational Feasibility

TECHNICAL FEASIBILITY

The HomeHunt system is technically feasible because it uses widely available technologies such as React, Django, and SQLite. These technologies are reliable, scalable, and capable of supporting large numbers of users and property listings. The system only requires a standard web server environment, making it easy to deploy and maintain.

ECONOMICAL FEASIBILITY

The proposed system is economically feasible since it is developed using open-source technologies, which significantly reduces development and operational costs. The system requires minimal investment in software tools and infrastructure while providing efficient rental management features for users and property owners.

BEHAVIOURAL FEASIBILITY

The HomeHunt system is behaviorally feasible because it provides a simple and user-friendly interface for both tenants and property owners. The platform simplifies the rental search process and encourages users to adopt the system by providing easy navigation and efficient search functionality.

OPERATIONAL FEASIBILITY

The system is operationally feasible due to its modular structure consisting of Admin, Owner, and User modules. Each module is designed to perform specific tasks that ensure efficient system operation. The platform allows users to easily search houses, owners to manage property listings, and administrators to monitor system activities.

3.1 PROPOSED SYSTEM

Considering the limitations of the existing system, the HomeHunt platform is proposed as a centralized web application that simplifies the process of rental house discovery and management. The system allows house owners to upload rental property details including location, rent amount, BHK type, floor type, and images. Tenants can search houses using filtering options based on district, place, price range, and other preferences.

The proposed system also provides features such as rental request submission, advance payment confirmation, complaint management, and feedback submission. By integrating these features into a single platform, HomeHunt improves the overall rental housing experience for both tenants and property owners. The system also includes a communication feature that allows tenants and property owners to interact through a chat interface. Once a rental request is sent by a user, both the user and the property owner can exchange messages regarding property details, availability, and advance payment confirmation. This feature improves communication and ensures faster decision making during the rental process.

3.1.1 JUSTIFICATION OF PROPOSED SYSTEM

The proposed system provides a structured and organized method for managing rental houses through a centralized platform. House owners can easily publish their property listings and manage tenant requests, while users can search and filter houses according to their preferences. The system ensures secure user authentication and efficient data management, making the rental search process faster and more reliable.

3.1.2 BENEFITS OF PROPOSED SYSTEM

- **Centralized platform for rental house discovery.**
- **Easy search and filtering of rental properties.**
- **Direct interaction between house owners and tenants.**
- **Secure rental request and advance payment system.**
- **Efficient complaint and feedback management.**

4. SYSTEM DESIGN

SYSTEM DESIGN

System Design is the process of designing the architecture, components, and interfaces for a system so that it meets the end-user requirements. The design phase transforms the requirements identified during system analysis into a detailed structure that can be implemented by developers. The system design for HomeHunt focuses on creating an efficient and user-friendly rental management platform where house owners can publish properties and tenants can easily discover rental houses.

A well-designed system must satisfy several important characteristics such as simplicity, flexibility, reliability, and efficiency. The design ensures that the system is capable of handling multiple users, managing property listings, processing rental requests, and maintaining secure data storage. The system also provides a real-time chat feature that enables tenants and property owners to communicate directly regarding rental requests and property details. The HomeHunt system design consists of logical design and physical design. Logical design describes the data flow, inputs, outputs, and system processes, while physical design focuses on the actual implementation of the system using programming languages, databases, and server infrastructure.

Characteristics of a well-designed system are:

- Acceptability
- Decision Making ability
- Economy
- Flexibility
- Reliability
- Simplicity

The design will determine the success of the system. System design is based on the information gathered during System analysis. System design goes through two phases of development: -

1. Logical Design - It describes the inputs, outputs, databases and procedures all in a format that meet the user's requirements.
2. Physical Design - This produces the working system by defining the design specification that tell programmers exactly what the candidate system must do.

4.1 INPUT DESIGN

Input design is the process of defining how data is entered into the system. It plays an important role in ensuring that accurate and valid data is provided by users. The HomeHunt system includes various input forms such as user registration forms, owner registration forms, house listing forms, and rental request forms.

The input forms are designed in a simple and user-friendly manner so that users can easily enter the required information. The system validates all input data to ensure correctness and completeness. For example, fields such as email addresses, contact numbers, and rent amounts are validated before being stored in the database. Proper input validation helps reduce errors and improves the overall reliability of the system.

.

4.2 OUTPUT DESIGN

Output design focuses on presenting processed data to users in a clear and understandable format. In the HomeHunt system, output information is displayed through various pages such as house listings, property details, request status pages, and administrative dashboards.

The system displays rental house information including location, rent amount, BHK type, and property images in a structured layout so that users can easily view and compare different houses. The output design ensures that all important information is presented clearly to improve user experience and assist users in making better rental decisions

The system also displays chat messages between users and property owners through a chat interface where both parties can communicate regarding rental requests.

.

UML DIAGRAM

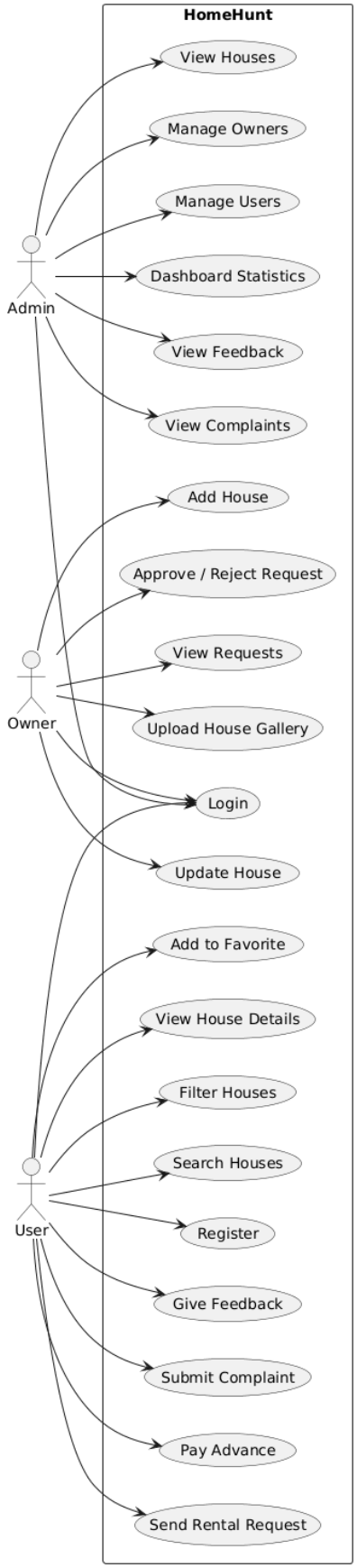
Unified Modeling Language (UML) diagrams are graphical representations used to visualize the structure and behavior of a system. UML diagrams help developers and stakeholders understand how different components of the system interact with each other.

In the HomeHunt system, UML diagrams are used to represent the relationship between different modules such as Admin, Owner, and User. These diagrams illustrate system workflows, data relationships, and interactions between system components. Common UML diagrams used in the system include Use Case Diagrams, Activity Diagrams, Class Diagrams, and Deployment Diagrams.

UML DIAGRAMS

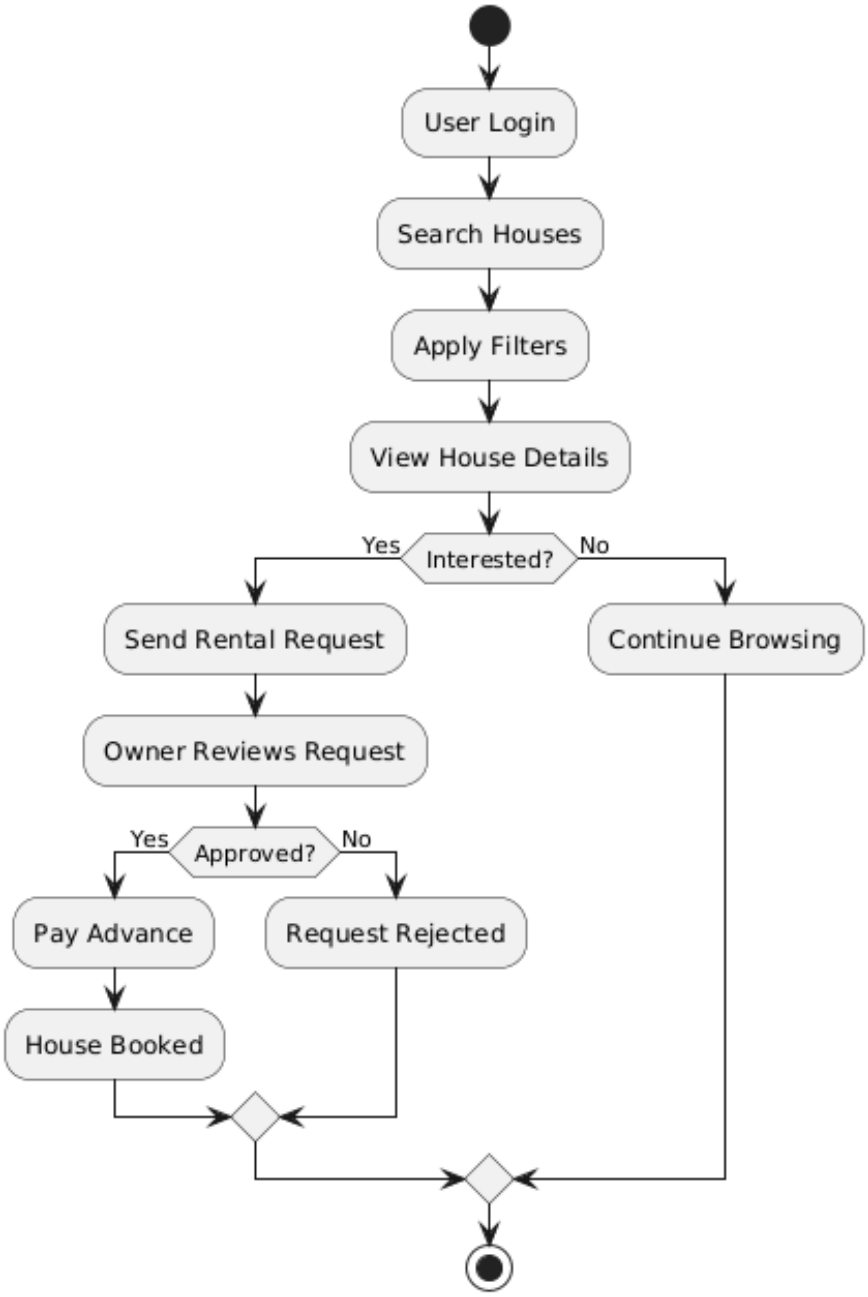
USE CASE DIAGRAM

HomeHunt -use case diagram



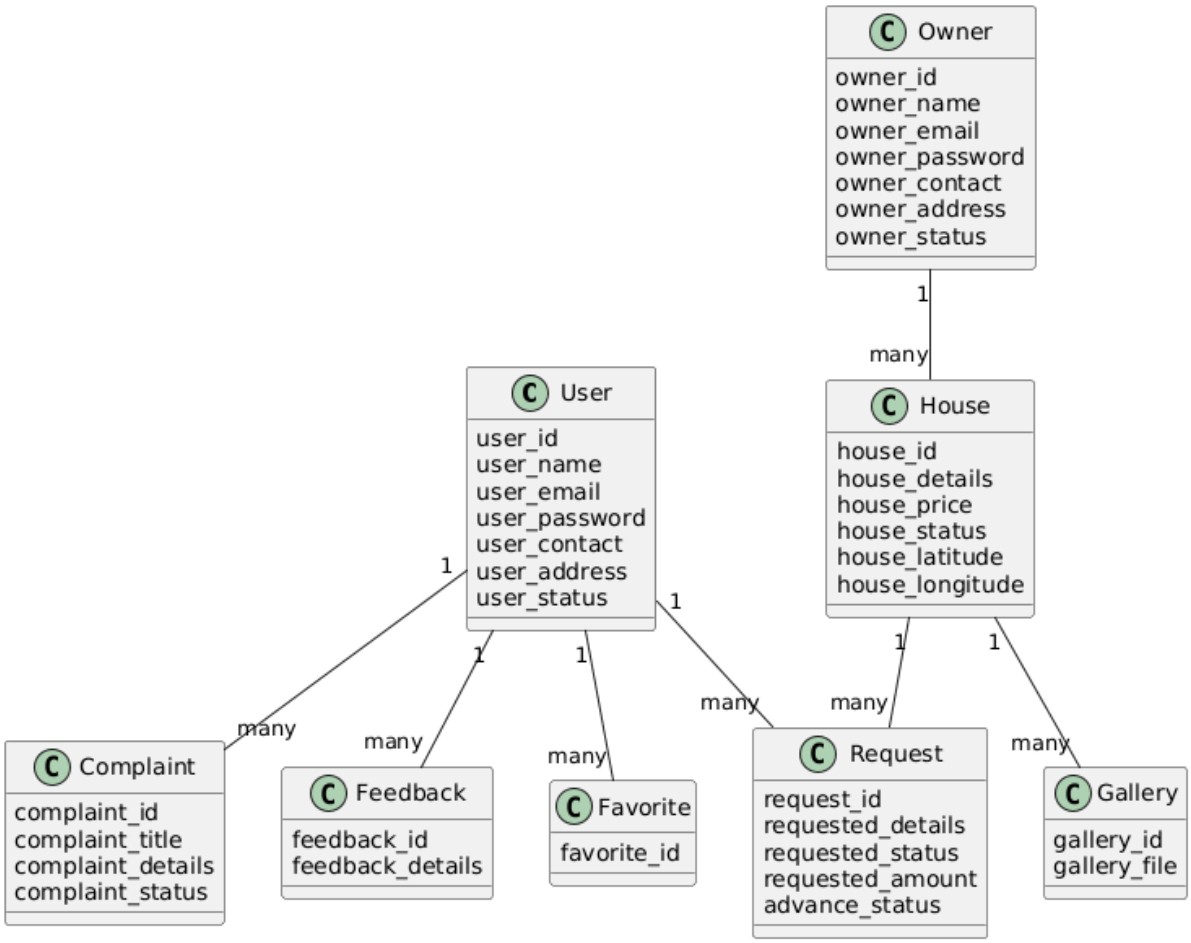
UML ACTIVITY DIAGRAM

HomeHunt - activity diagram



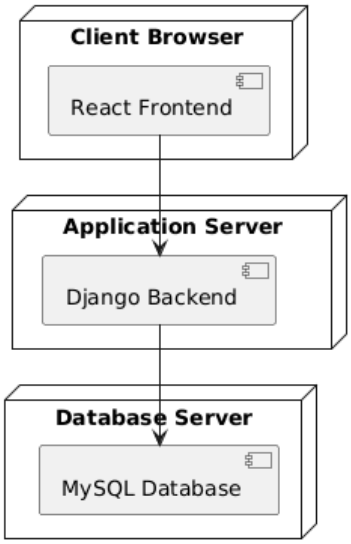
UML CLASS DIAGRAM

HomeHunt - class diagram



UML DEPLOYMENT DIAGRAM

HomeHunt- deployment diagram



4.3 TABLE DESIGN

Database design is an essential part of the HomeHunt system because it stores all important information related to users, property owners, houses, and rental requests. A well-designed database ensures efficient storage and retrieval of data while maintaining data consistency and security.

The database consists of multiple tables that store information about administrators, owners, users, houses, gallery images, rental requests, payments, complaints, and feedback. Each table is designed with appropriate primary keys and foreign keys to establish relationships between different data entities. Proper database design ensures efficient data management and improves the performance of the system.

TABLES

TABLE 1

- **Table Name**: tbl_admin
- **Primary Key**: admin_id
- **Foreign Keys:** None
- **Table Description**: Contains administrator details

Field	DataType	Size	Constraints	Field Description
admin_id	INT	11	Primarykey	Admin ID
admin_name	VARCHAR	50	NOTNULL	Admin name
admin_email	VARCHAR	50	NOTNULL	Admin email
admin_password	VARCHAR	50	NOT NULL	Admin password

TABLE 2

- **Table Name**: tbl_bhktype
- **Primary Key**: bhktype_id
- **Foreign Keys:** None
- **Table Description**: Contains BHK types

Field	DataType	Size	Constraints	Field Description
bhktype_id	INT	11	Primary key	BHK Type ID
bhktype_name	varchar	50	NOT NULL	BHK type name

TABLE 3

- **Table Name**: tbl_house
- **Primary Key**: house_id
- **ForeignKeys:**bhktype→tbl_bhktype

  floortype→tbl_floortype

  place→tbl_place

  owner→tbl_owner

  tenanttype → tbl_tenenttype

- **Table Description**: Contains house details uploaded by property owners

Field	**DataType**	**Size**	**Constraints**	**Field Description**
house_id	INT	11	Primarykey	House ID
house_photo	VARCHAR	200	NOT NULL	House photo
house_details	TEXT		NOT NULL	House description
house_price	INT	11	NOTNULL	House rent
bhktype_id	INT	11	FOREIGN KEY	BHK type
floortype_id	INT	11	FOREIGN KEY	Floor type
place_id	INT	11	FOREIGN KEY	Place
owner_id	INT	11	FOREIGN KEY	Owner
tenanttype_id	INT	11	FOREIGN KEY	Tenant type

house_latitude	FLOAT		NULL	Location latitude
house_longitude	FLOAT		NULL	Location longitude
house_status	VARCHAR	20	DEFAULT Active	House status

TABLE 4

- **Table Name**: tbl_requested
- **Primary Key**: request_id
- **Foreign Keys:** user_id → tbl_user(user_id)
  house_id → tbl_house(house_id)
- **Table Description**: Stores house rental requests sent by users to owners.

Field	**DataType**	**Size**	**Constraints**	**Field Description**
request_id	INT	11	Primary key	Request ID
user_id	INT	11	FOREIGN KEY	User ID
House_id	INT	11	FOREIGN KEY	Requested house
requested_details	VARCHAR	200	NOT NULL	Request message
requested_reply	VARCHAR	200	NULL	Owner reply to request
requested_status	VARCHAR	20	NOT NULL	Request status
requested_amount	INT	11	NOTNULL	Advance payment amount
advance_status	VARCHAR	20	DEFAULT Not Paid	Advance payment status

TABLE 5

- **Table Name**: tbl_complaint
- **Primary Key**: complaint_id
- **Foreign Keys:** user_id → tbl_user(user_id)
- **Table Description**: Contains complaint details

Field	**DataType**	**Size**	**Constraints**	**Field Description**
complaint_id	INT	11	Primarykey	Complaint ID
complaint_status	VARCHAR	20	NOT NULL	Complaint status
complaint_title	VARCHAR	60	NOTNULL	Complaint title
complaint_details	VARCHAR	60	NOTNULL	Complaint details
complaint_reply	VARCHAR	50	NOTNULL	Complaint reply
user_id	INT	11	NOTNULL	User ID

TABLE 6

- **Table Name**: tbl_owner
- **PrimaryKey**: owner_id
- **Foreign Keys:** None
- **Table Description**: owners who can upload and manage rental houses

Field	Data Type	Size	Constraints	Field Description
owner_id	INT	11	PRIMARY KEY	Owner ID
owner_name	VARCHAR	100	NOT NULL	Owner name
owner_email	VARCHAR	100	NOT NULL	Owner email
owner_password	VARCHAR	100	NOT NULL	Owner password
owner_contact	VARCHAR	15	NULL	Owner contact
owner_address	TEXT		NOT NULL	Owner address
owner_photo	VARCHAR	200	NULL	Owner photo
owner_proof	VARCHAR	200	NULL	Owner proof
owner_doj	DATE		NULL	Date of joining
owner_status	VARCHAR	20	DEFAULT Active	Owner status

TABLE 7

- **Table Name**: tbl_feedback
- **Primary Key**: feedback_id
- **Foreign Keys:** user_id → tbl_user(user_id)
- **Table Description**: Contains feedback details

Field	Data Type	Size	Constraints	Field Description
feedback_id	INT	11	PRIMARY KEY	Feedback ID
user_id	INT	11		User ID
feedback_details	TEXT			Feedback content

TABLE 8

- **Table Name**: tbl_gallery
- **Primary Key**: gallery_id
- **Foreign Keys:** house_id → tbl_house
- **Table Description**: Contains Houses gallery details

Field	Data Type	Size	Constraints	Field Description
gallery_id	INT	11	PRIMARY KEY	Gallery ID
gallery_file	VARCHAR	50	NOT NULL	Gallery image
house_id	INT	11	FOREIGN KEY	House ID

TABLE 9

Table Name: tbl_district

- **Primary Key**: district_id
- **Foreign Keys:** None
- **Table Description**: Contains district details

Field	Data Type	Size	Constraints	Field Description
district_id	INT	11	PRIMARY KEY	District ID
district_name	VARCHAR	50	NOT NULL	District name

TABLE 10

- **Table Name**: tbl_floortype
- **Primary Key**: floortype_id
- **Foreign Keys:** none
- **Table Description**: Contains floor type details

Field	Data Type	Size	Constraints	Field Description
floortype_id	INT	11	PRIMARY KEY	Floor Type ID
floortype_name	VARCHAR	50	NOT NULL	Floor type name

TABLE 11

- **Table Name**: tbl_tenenttype
- **Primary Key**: tenanttype_id
- **Foreign Keys:** None
- **Table Description:** Contains tenant type details

Field	Data Type	Size	Constraints	Field Description
tenanttype_id	INT	11	PRIMARY KEY	Tenant Type ID
tenanttype_name	VARCHAR	50	NOT NULL	Tenant type name

TABLE 12

- **Table Name**: tbl_place
- **Primary Key**: place_id
- **Foreign Keys:** district_id → tbl_district(district_id)
- **Table Description**: Contains place details

Field	Data Type	Size	Constraints	Field Description
place_id	INT	11	PRIMARY KEY	Place ID
place_name	VARCHAR	50	NOT NULL	Place name
district_id	INT	11		District ID

TABLE 13

- **Table Name**: tbl_favorite
- **Primary Key**: favorite_id
- **Foreign Keys:** user_id → tbl_user(user_id)
  house_id → tbl_house(house_id)
- **Table Description:** Stores the houses marked as favorite by users.

Field	Data Type	Size	Constraints	Field Description
favorite_id	INT	11	PRIMARY KEY	Favorite ID
user_id	INT	11	FOREIGN KEY	User who marked the house
house_id	INT	11	FOREIGN KEY	Favorite house

TABLE 14

- **Table Name**: tbl_user
- **Primary Key**: user_id
- **Foreign Keys:** tenanttype → tbl_tenenttype(tenanttype_id)
- **Table Description:** Stores user account details and tenant information

Field	Data Type	Size	Constraints	Field Description
user_id	INT	11	PRIMARY KEY	User ID
user_name	VARCHAR	50	NOT NULL	User name
user_email	VARCHAR	50	NOT NULL	User email
user_password	VARCHAR	50	NOT NULL	User password
user_photo	VARCHAR	200	NULL	User photo
user_proof	VARCHAR	200	NULL	User proof
user_address	VARCHAR	200	NOT NULL	User address
user_dob	VARCHAR	20	NOT NULL	Date of birth
user_contact	VARCHAR	15	NULL	Contact number
tenanttype_id	INT	11	FOREIGN KEY	Tenant type
user_status	VARCHAR	20	DEFAULT Active	User status

TABLE 15

- **Table Name**: tbl_userfloortype
- **Primary Key**: userfloortype_id

- **Foreign Keys:** floortype_id → tbl_floortype(floortype_id)
  • user_id → tbl_user(user_id)

- **Table Description:** Stores the preferred floor types selected by users.

Field	Data Type	Size	Constraints	Field Description
userfloortype_id	INT	11	PRIMARY KEY	User floor preference ID
floortype_id	INT	11	FOREIGN KEY	Floor type selected by user
user_id	INT	11	FOREIGN KEY	User ID

TABLE 16

- **Table Name**: tbl_userbhktype
- **Primary Key**: userbhktype_id

- **Foreign Keys:** bhktype_id → tbl_bhktype(bhktype_id)
  • user_id → tbl_user(user_id)

- **Table Description:** Stores the preferred BHK types selected by users.

Field	Data Type	Size	Constraints	Field Description
userbhktype_id	INT	11	PRIMARY KEY	User BHK preference ID
bhktype_id	INT	11	FOREIGN KEY	BHK type selected by user
user_id	INT	11	FOREIGN KEY	User ID

TABLE 17

Table Name: tbl_chat

- **Primary Key**: chat_id
- **Foreign Keys:** request_id → tbl_requested(request_id)
- **Table Description**: Stores chat messages between users and house owners related to rental requests.

Field	Data Type	Size	Constraints	Field Description
chat_id	INT	11	PRIMARY KEY	Chat message ID
request_id	INT	11	FOREIGN KEY	Related rental request
sender	VARCHAR	20	NOT NULL	Message sender
sender_id	INT	11	NOT NULL	ID of message sender
message	TEXT		NOT NULL	Chat message
created_at	DATETIME		DEFAULT CURRENT_TIMESTAMP	Message time

5. SYSTEM TESTING

SYSTEM TESTING

System Testing is a type of software testing performed on a complete integrated system to evaluate its compliance with specified requirements. In HomeHunt, the system is divided into three primary modules - Admin, Owner, and User - each tested separately before integration.

METHODS ADOPTED FOR TESTING

Testing is one of the major hurdles in the development of the system. Testing is the process of finding errors in the system. Only error free software will be stable for a long time.

There are different types of techniques for finding the bugs in software:

Syntax Testing:

We use syntax testing on HomeHunt to eliminate errors in software. In the system we have input fields like text, file and numeric fields. We should allow only integer value in numeric and should avoid any alphabets. The program won't accept any alphabets in a numeric field (e.g.: User details in registration) and also won't accept any numeric in alphabetic field (e.g.: name details in registration). It also check whether there is input in each field before insertion.

System Testing:

System testing involves unit testing, integration testing, acceptance testing. Careful planning and scheduling are required to ensure that modules will be available for integration into the evolving software product when needed. A test plan has the following steps:

1. Prepare test plan
2. Specify conditions for user acceptance testing
3. Prepare test data for program testing
4. Prepare test data for transaction path testing
5. Plan user training
6. Compile/assemble programs
7. Prepare job performance aids
8. Prepare operational documents

Unit Testing:

Unit testing involves testing different modules in the system. A system is made up of several modules, modules are assembled and integrated to perform a specific function. Unit testing focus on testing modules independently of one another to locate errors in coding and logic that are contained within each module. This enables to recognize errors more easily thus making debugging easier. Focused on testing modules (Admin, Owner, User) independently.

Integration Testing:

Where different units or components of an application are tested together to ensure they work correctly as a group. The goal is to identify any issues or bugs that may arise when individual pieces of the software interact with each other. bottom-up integration, top-down integration and sandwich integration strategy these are different integration testing strategies. Here we used bottom-up strategy for HomeHunt. In bottom-up strategy testing begins with lower level modules and moves up towards the higher-level ones.

Acceptance Testing:

It is where the system is evaluated to ensure it meets the user's needs and requirements. It is typically performed after system testing and is focused on validating the product from the end user's perspective. The goal is to ensure the software works as intended in a real-world scenario and is ready for delivery or release. Functional test causes involve exercising the code with nominal files for which the expected outputs are known. Thus, the software system HomeHunt developed in the above manner is one that satisfies the user needs, confirms to it.

TEST RESULTS

- HomeHunt is an innovative platform designed to streamline the connection between house owners and tenants. This comprehensive system combines intuitive property management, user interaction, and efficient rental request processing to create a seamless experience. With its user-friendly interface and robust features, HomeHunt helps property owners publish their rental houses effectively while enabling users to easily search, view, and request suitable houses through the platform.

The main benefits of the system are:

- Improved rental house discovery and property visibility.

- More efficient communication between tenants and house owners.
- Enhanced user experience through advanced search and filtering options.
- Increased satisfaction for both tenants and property owners.
- Reduction in manual effort for property owners to manage rental listings.
- Improved management of rental requests and advance payments.
- Better organization of rental house information in a centralized platform.
- Simplified process for tenants to find suitable houses quickly.
- Real-time communication between tenants and property owners through chat messaging.

6. SYSTEM SECURITY

SYSTEM SECURITY

HomeHunt, as a platform handling user information and property data, implements a reliable security mechanism to protect the system from unauthorized access and misuse. Security technology ensures that the protection mechanisms built into the system safeguard the application from improper penetration and data manipulation.

The system ensures the confidentiality and integrity of user data by restricting access to registered users only. During the registration process, users and property owners are required to provide valid email addresses and passwords. These credentials are used for authentication during login, ensuring that only authorized users can access the system. Based on the login credentials, the system identifies the role of the user such as admin, owner, or tenant and provides access accordingly.

The HomeHunt system also ensures that property owners can manage only their own house listings while administrators have full control over the platform. Data validation techniques are applied to input fields to prevent incorrect or malicious data from entering the system. Sensitive information such as user credentials is stored securely in the database.

File uploads such as house images and profile pictures are validated to ensure only permitted file types are uploaded to the system. Additionally, proper session management techniques are implemented to maintain secure user sessions during system usage. These security measures help maintain the reliability, privacy, and integrity of the HomeHunt platform.

7. SYSTEM IMPLEMENTATION AND MAINTENANCE

SYSTEM IMPLEMENTATION AND MAINTENANCE

System implementation is the process of bringing the developed system into operational use. The implementation of the HomeHunt system follows a structured process that begins with system planning, requirement analysis, and development of core functionalities such as user registration, authentication, and property management.

A significant stage in the implementation process is the development of the owner module, which allows property owners to upload and manage rental house listings. This module enables owners to add house details including rent amount, BHK type, location, and property images. The system also integrates the tenant module, which allows users to search houses, view detailed property information, and send rental requests.

After completing the development phase, the system is deployed in the server environment where it becomes accessible to users through a web browser. Security measures such as user authentication, role-based access control, and data validation are implemented to ensure system reliability and data protection. Continuous monitoring and maintenance are required to ensure the system functions efficiently and remains secure over time.

Maintenance activities include correcting system errors, improving system performance, updating software components, and adding new features when required. These activities help maintain the stability and efficiency of the HomeHunt system.

.

8. SCOPE FOR FUTURE ENHANCEMENT

SCOPE FOR FUTURE ENHANCEMENT

HomeHunt's modular design provides significant potential for future enhancements. Key areas for potential expansion include:

- Integration of a secure online payment gateway for rental advance payments.
- Implementation of a recommendation system to suggest rental houses to users based on their search preferences.
- Development of native mobile applications for Android and iOS platforms.
- Integration of map-based location services to display property locations accurately.
- Introducing an online rental agreement generation feature for secure documentation.
- Implementing advanced analytics for property owners to track property views and rental requests.
- Enhancement of real-time chat with notifications and message status indicators.

9. CONCLUSION

CONCLUSION

HomeHunt provides an efficient and user-friendly solution for rental house discovery and property management through a centralized digital platform. The system simplifies the process of searching rental houses by allowing tenants to browse available properties, filter houses based on their preferences, and communicate directly with property owners.

The platform enables property owners to publish and manage house listings efficiently while administrators maintain overall system control. Through its structured design and modern web technologies, HomeHunt improves communication between tenants and property owners and provides an organized approach to rental property management.

By digitizing the rental housing process, the HomeHunt system reduces the time required to find suitable houses and improves accessibility to rental property information. The system demonstrates how modern web technologies can be used to solve real-world housing challenges and provide practical solutions for both property owners and tenants.

.

10. BIBLIOGRAPHY

BIBLIOGRAPHY

1. https://react.dev
2. www.geek-university.com
3. www.djangoproject.com
4. www.reactjs.org
5. www.sqlite.org
6. www.w3schools.com
7. www.geeksforgeeks.org
8. developer.mozilla.org
9. stackoverflow.com
10. www.tutorialspoint.com
11. www.freecodecamp.org
12. www.digitalocean.com
13. www.codeproject.com
14. www.medium.com
15. www.researchgate.net
16. www.github.com
17. javascript.info
18. www.css-tricks.com
19. www.python.org
20. www.djangoproject-rest-framework.org
21. www.programiz.com
22. www.javatpoint.com

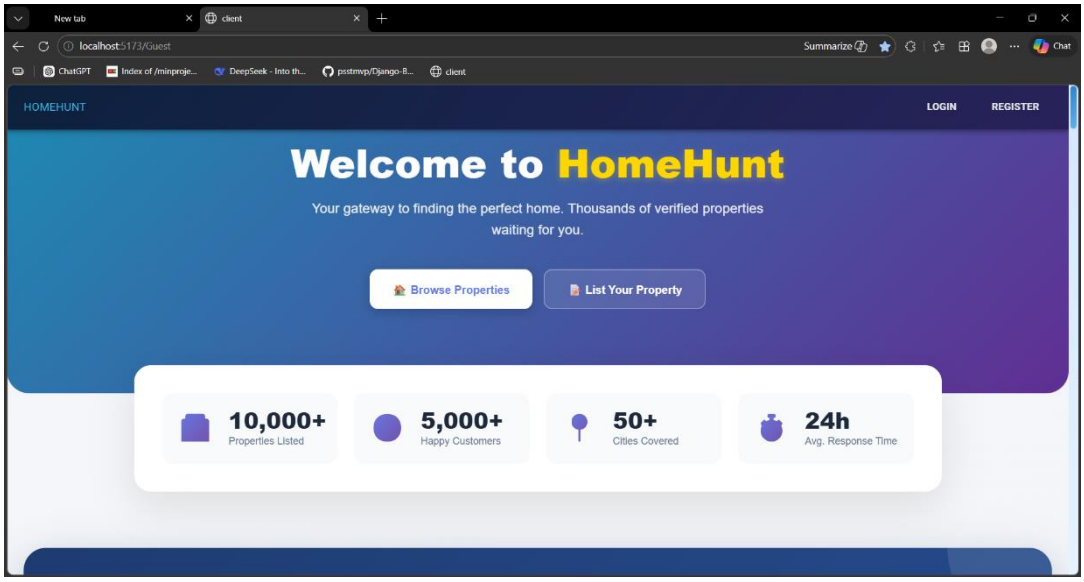
.

11. APPENDIX

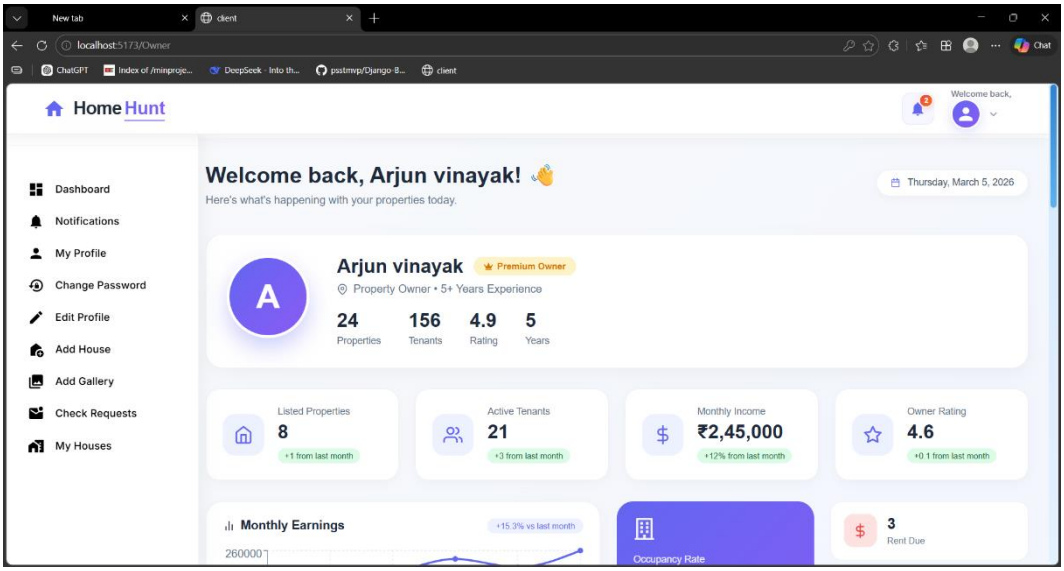
APPENDIX

11.1 SCREENSHOTS

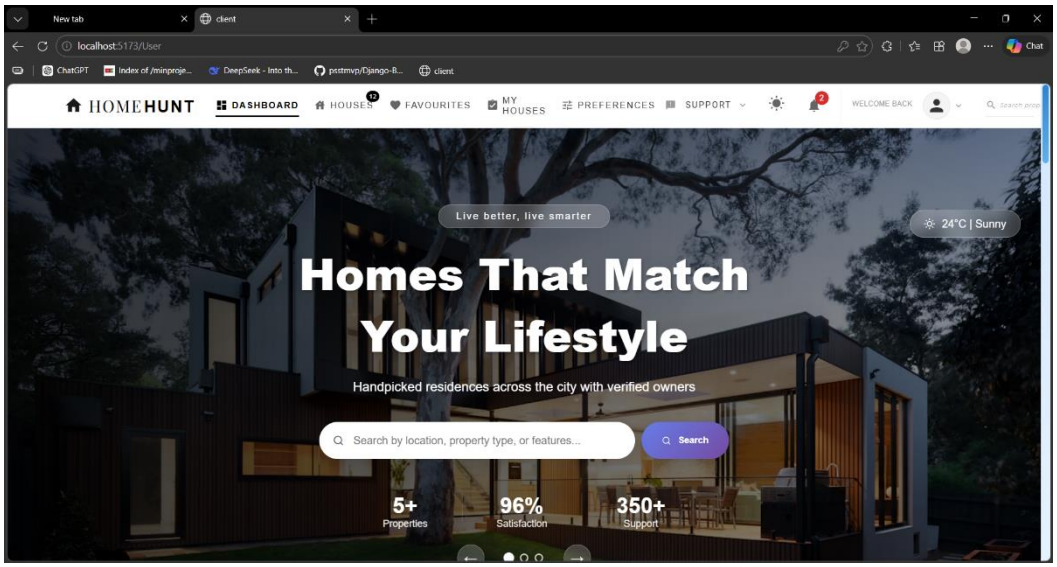
LANDINGPAGE



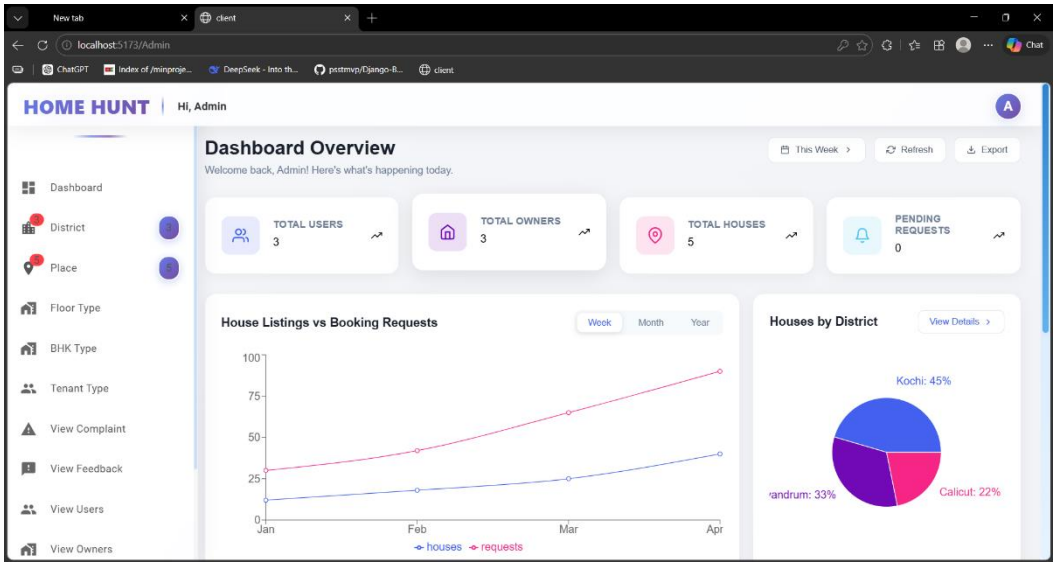
OWNER DASHBOARD



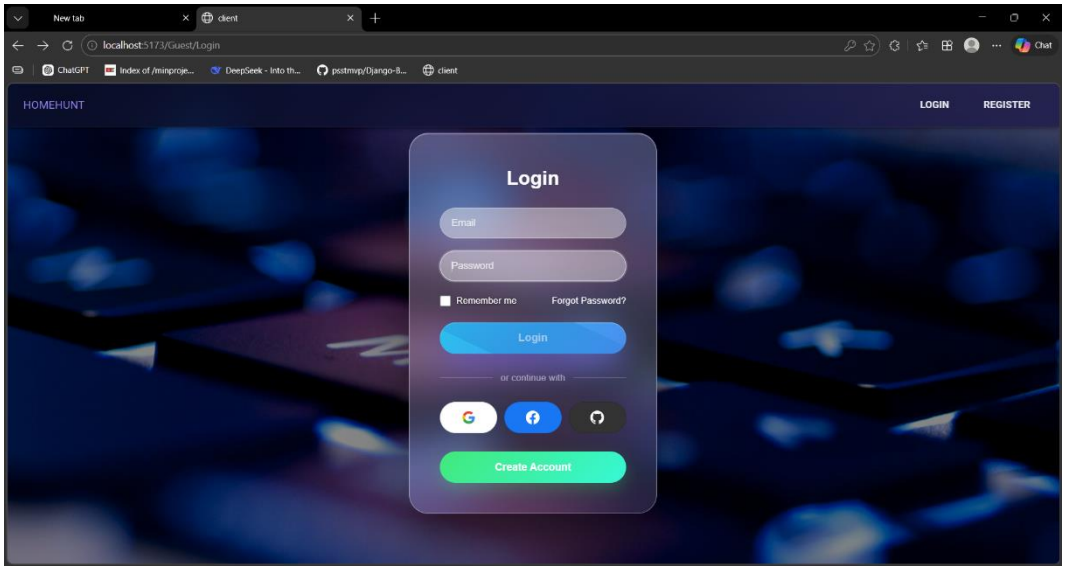
USER DASHBOARD



ADMIN DASHBOARD



LOGINPAGE


New tab
client
localhost:5173/Guest/Login
ChatGPT
Index of /minproje...
DeepSeek - Into th...
psstmvp/Django-B...
client
Chat
HOMEHUNT
LOGIN
REGISTER
Login
Email
Password
Remember me
Forgot Password?
Login
or continue with
Create Account

- **CODING**

LOGIN PAGE

```
import Styles from "./Login.module.css";
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-
dom';

const Login = () => {
   const [email, setEmail] = useState('');
   const [password, setPassword] =
useState('');
   const navigate = useNavigate();

   const handleLogin = () => {
      axios.post('http://127.0.0.1:8000/Login/',
{ email, password })
         .then(res => {
            const { role, id, name, message } =
res.data;
            alert(message);
            // route by role
            if (role === 'user') {
               sessionStorage.setItem('uid', id);
               sessionStorage.setItem('userName'
, name);
               navigate('/User');
            } else if (role === 'admin') {
```

```
                sessionStorage.setItem('aid', id);
                sessionStorage.setItem('adminNa
me', name);
                navigate('/Admin');
            } else if (role === 'owner') {
                sessionStorage.setItem('oid', id);
                sessionStorage.setItem('ownerNa
me', name);
                navigate('/Owner');
            } else {
                alert('Unknown role');
            }
          })
          .catch(err => alert('Login failed'));
    };
  return (
      <div
className={Styles.logincontainer}>
          <div className={Styles.loginBox}>
            <h2
className={Styles.title}>Login</h2>

            {/* Fixed input fields - using div
instead of table elements */}
            <div
className={Styles.inputGroup}>
              <div
className={Styles.inputField}>
                <input
                  type="email"
```

```
                    placeholder="Email"
                    className={Styles.input}
                    value={email}
                    onChange={e =>
setEmail(e.target.value)}
                  />
                </div>
                <div
className={Styles.inputField}>
                  <input
                    type="password"
                    placeholder="Password"
                    className={Styles.input}
                    value={password}
                    onChange={e =>
setPassword(e.target.value)}
                  />
                </div>
              </div>

              <div className={Styles.options}>
                <label>
                  <input type="checkbox" />
Remember me
                </label>
                <a href="#">Forgot
Password?</a>
              </div>

              <button className={Styles.button}
onClick={handleLogin}>Login</button>
```

```
            {/* DIVIDER */}
            <div className={Styles.divider}>
               <span>or continue with</span>
            </div>

            {/* SOCIAL LOGIN */}
            <div
className={Styles.socialLogin}>
               <button
                  className={`${Styles.socialBt
n} ${Styles.googleBtn}`}
                  type="button"
                  onClick={() =>
window.open("https://accounts.google.com/",
"_blank")}
               >
                  <svg width="20" height="20"
viewBox="0 0 24 24">
                     <path fill="#4285F4"
d="M22.56 12.25c0-.78-.07-1.53-.2-
2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21
3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-
8.09z" />
                     <path fill="#34A853"
d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-
2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-
1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23
12 23z" />
                     <path fill="#FBBC05"
d="M5.84 14.09c-.22-.66-.35-1.36-.35-
```

```
2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55
1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-
.62z" />
                    <path fill="#EA4335"
d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-
3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47
2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-
4.53z" />
                </svg>
            </button>
            <button
                className={`${Styles.socialBt
n} ${Styles.facebookBtn}`}
                type="button"
                onClick={() =>
window.open("https://www.facebook.com/logi
n/", "_blank")}
            >
                <svg width="20" height="20"
viewBox="0 0 24 24">
                    <path fill="#FFFFFF"
d="M24 12.073c0-6.627-5.373-12-12-12s-12
5.373-12 12c0 5.99 4.388 10.954 10.125
11.854v-8.385H7.078v-3.47h3.047V9.43c0-
3.007 1.792-4.669 4.533-4.669 1.312 0
2.686.235 2.686.235v2.953H15.83c-1.491 0-
1.956.925-1.956 1.874v2.25h3.328l-.532
3.47h-2.796v8.385C19.612 23.027 24 18.062
24 12.073z" />
                </svg>
            </button>
```

```
                <button
                    className={`${Styles.socialBt
n} ${Styles.githubBtn}`}
                    type="button"
                    onClick={() =>
window.open("https://github.com/login",
"_blank")}
                >
                    <svg width="20" height="20"
viewBox="0 0 24 24">
                        <path fill="#FFFFFF"
d="M12 0c-6.626 0-12 5.373-12 12 0 5.302
3.438 9.8 8.207 11.387.599.111.793-.261.793-
.577v-2.234c-3.338.726-4.033-1.416-4.033-
1.416-.546-1.387-1.333-1.756-1.333-1.756-
1.089-.745.083-.729.083-.729 1.205.084 1.839
1.237 1.839 1.237 1.07 1.834 2.807 1.304
3.492.997.107-.775.418-1.305.762-1.604-
2.665-.305-5.467-1.334-5.467-5.931 0-
1.311.469-2.381 1.236-3.221-.124-.303-.535-
1.524.117-3.176 0 0 1.008-.322 3.301
1.23.957-.266 1.983-.399 3.003-.404 1.02.005
2.047.138 3.006.404 2.291-1.552 3.297-1.23
3.297-1.23.653 1.653.242 2.874.118
3.176.77.84 1.235 1.911 1.235 3.221 0 4.609-
2.807 5.624-5.479 5.921.43.372.823 1.102.823
2.222v3.293c0 .319.192.694.801.576 4.765-
1.589 8.199-6.086 8.199-11.386 0-6.627-
5.373-12-12-12z" />
                    </svg>
                </button>
```

```
            </div>

            {/* CREATE ACCOUNT BUTTON
*/}
            <button
classNameName={`${Styles.registerBtn}
${Styles.createAccountBtn}`}>
                Create Account
            </button>
          </div>
        </div>
    );
};

export default Login;
```

USER REGISTRATIONPAGE

```
 import React, { useState, useEffect } from
"react";
 import { useNavigate } from "react-router-
dom";
 import Styles from
"./Registeration.module.css";
 import axios from "axios";

 const Registeration = () => {
   const [form, setForm] = useState({
     name: "",
     email: "",
     password: "",
     contact: "",
     address: "",
     dob: "",
     tenenttype: "",
     photo: null,
     proof: null
   });
   const navigate = useNavigate();

   const [tenentTypes, setTenentTypes] =
useState([]);
   const [fileName, setFileName] =
useState("No file chosen");
   const [proofFileName, setProofFileName] =
useState("No file chosen");
```

```
useEffect(() => {
  loadTenentTypes();
}, []);

const loadTenentTypes = () => {
  axios
    .get("http://127.0.0.1:8000/tenenttype/")
    .then((res) => {
      setTenentTypes(res.data.data || []);
    })
    .catch((err) => {
      console.log(err);
    });
};

const handleChange = (e) => {
  const { name, value } = e.target;
  setForm({
    ...form,
    [name]: value
  });
};

const handleFileChange = (e) => {
  const file = e.target.files[0];
  if (file) {
    setForm({
      ...form,
      photo: file
    });
    setFileName(file.name);
```

```
    }
  };
  const handleProofChange = (e) => {
    const file = e.target.files[0];
    if (file) {
      setForm({
        ...form,
        proof: file
      });
      setProofFileName(file.name);
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();

    const formData = new FormData();
    formData.append("user_name",
form.name);
    formData.append("user_email",
form.email);
    formData.append("user_password",
form.password);
    formData.append("user_contact",
form.contact);
    formData.append("user_address",
form.address);
    formData.append("user_dob", form.dob);
    formData.append("tenenttype_id",
form.tenenttype);
    formData.append("user_photo",
```

```
form.photo);
    formData.append("user_proof",
form.proof);

    console.log(formData);

    axios
      .post("http://127.0.0.1:8000/User/",
formData)
      .then(() => {
        alert("User Registered Successfully");
          navigate("/Guest/Login"); // Redirect to
login after successful registration
        setForm({
          name: "",
          email: "",
          password: "",
          contact: "",
          address: "",
          dob: "",
          tenenttype: "", // Fixed: consistent with
state key
          photo: null,
          proof: null
        });
        setFileName("No file chosen");
        setProofFileName("No file chosen");
      })
      .catch((error) => {
        console.log(error);
        alert("Registration Failed");});};
```

BACKEND DATABASE MODELS (DJANGO)

```
from django.db import models

# Create your models here.

class tbl_district(models.Model):
    district_name=models.CharField(max_lengt
h=50)

class tbl_admin(models.Model):
    admin_name=models.CharField(max_lengt
h=50)
    admin_email=models.CharField(max_lengt
h=50)
    admin_password=models.CharField(max_le
ngth=50)
class tbl_bhktype(models.Model):
    bhktype_name=models.CharField(max_leng
th=50)
class tbl_floortype(models.Model):
    floortype_name=models.CharField(max_len
gth=50)
class tbl_tenenttype(models.Model):
    tenanttype_name=models.CharField(max_le
ngth=50)
class tbl_user(models.Model):
    user_name =
models.CharField(max_length=50)
    user_email =
models.CharField(max_length=50)
    user_password =
models.CharField(max_length=50)
    user_photo = models.FileField(
        upload_to='Assets/UserPhoto/',
```

```
        null=True,
        blank=True
    )
    user_proof = models.FileField(
        upload_to='Assets/UserProof/',
        null=True,
        blank=True
    )
    user_address =
models.CharField(max_length=200)
    user_dob =
models.CharField(max_length=20)
    user_contact = models.CharField(
        max_length=15,
        blank=True,
        null=True)
    tententtype =
models.ForeignKey(tbl_tenenttype,
on_delete=models.CASCADE)
    user_status = models.CharField(
        max_length=20,
        default="Active"   # Active | Blocked |
Inactive
    )

class tbl_place(models.Model):
    place_name=models.CharField(max_length
=50)
    district=models.ForeignKey(tbl_district,on_
delete=models.CASCADE)
class tbl_owner(models.Model):
    owner_name =
models.CharField(max_length=100)
    owner_email =
models.CharField(max_length=100)
    owner_password =
models.CharField(max_length=100)
```

```
    owner_contact =
models.CharField(max_length=15,
blank=True, null=True)  #
    owner_address = models.TextField()
    owner_photo =
models.ImageField(upload_to="owner_photo/
", blank=True, null=True)
    owner_proof =
models.ImageField(upload_to="owner_proof/"
, blank=True, null=True)
    owner_doj = models.DateField(blank=True,
null=True)

    owner_status = models.CharField(
        max_length=20,
        default="Active"    # Active | Blocked |
Inactive
    )

class tbl_house(models.Model):
    house_photo =
models.FileField(upload_to='Assets/HousePho
to/')
    house_details = models.TextField()
    house_price = models.IntegerField()

    bhktype =
models.ForeignKey('tbl_bhktype',
on_delete=models.CASCADE)
    floortype =
models.ForeignKey('tbl_floortype',
on_delete=models.CASCADE)
    place = models.ForeignKey('tbl_place',
on_delete=models.CASCADE)
    owner = models.ForeignKey('tbl_owner',
on_delete=models.CASCADE)
    tenanttype =
```

```
models.ForeignKey('tbl_tenenttype',
on_delete=models.CASCADE)
    house_langtitude =
models.FloatField(null=True, blank=True)
    house_longtitude =
models.FloatField(null=True, blank=True)

    house_status = models.CharField(
        max_length=20,
        default="Active"
    )

    def __str__(self):
        return f"{self.bhktype} - {self.place}"
class tbl_userfloortype(models.Model):
    floortype_id=models.ForeignKey(tbl_floort
ype,on_delete=models.CASCADE)
    user_id=models.ForeignKey(tbl_user,on_de
lete=models.CASCADE)
class tbl_userbhktype(models.Model):
    bhktype_id=models.ForeignKey(tbl_bhktyp
e,on_delete=models.CASCADE)
    user_id=models.ForeignKey(tbl_user,on_de
lete=models.CASCADE)
class tbl_gallery(models.Model):
    house_id=models.ForeignKey(tbl_house,on
_delete=models.CASCADE)
    gallery_file=models.FileField(upload_to='A
ssets/Gallery/')
class tbl_favorite(models.Model):
    user_id=models.ForeignKey(tbl_user,on_de
lete=models.CASCADE)
    house_id=models.ForeignKey(tbl_house,on
_delete=models.CASCADE)
class tbl_requested(models.Model):
    user_id=models.ForeignKey(tbl_user,on_de
lete=models.CASCADE)
```

```
    house_id=models.ForeignKey(tbl_house,on
_delete=models.CASCADE)
    requested_details=models.CharField(max_l
ength=200)
    requested_reply=models.CharField(max_len
gth=200)
    requested_status=models.CharField(max_le
ngth=20)
    requested_amount = models.IntegerField()
    advance_status = models.CharField(
        max_length=20,
        default="Not Paid" )
class tbl_complaint(models.Model):
    user_id=models.ForeignKey(tbl_user,on_de
lete=models.CASCADE)
    complaint_title=models.CharField(max_len
gth=100)
    complaint_details=models.CharField(max_l
ength=200)
    complaint_reply=models.CharField(max_le
ngth=200)
    complaint_status=models.CharField(max_le
ngth=20)
class tbl_feedback(models.Model):
    user_id=models.ForeignKey(tbl_user,on_de
lete=models.CASCADE)
    feedback_details=models.CharField(max_le
ngth=200)
class tbl_rentpayment(models.Model):

    house_id = models.ForeignKey(
        tbl_house,
        on_delete=models.CASCADE
    )

    user_id = models.ForeignKey(
        tbl_user,
```

```
        on_delete=models.CASCADE
    )

    rent_month =
models.CharField(max_length=20)

    rent_amount = models.IntegerField()

    payment_status = models.CharField(
        max_length=20,
        default="Paid"
    )

    payment_date = models.DateTimeField(
        auto_now_add=True
    )
```